

문서와 논문을 요약하는 AI 연구원

학교나 대학원에서 공부하는 학생이나 직장인은 여러 문서를 빠르게 읽고 요약해서 필요한 정보를 파악하는 일이 많습니다. 이번 장에서는 다양한 문서를 정해진 양식으로 요약하는 방법을 살펴보겠습니다.

최근에 문서를 요약해 주는 상용 서비스가 속속 등장하고 있습니다. 물론 이런 서비스를 이용해도 되지만, 서비스를 직접 개발하면서 어떤 원리로 동작하는지 파악한다면 대규모 언어 모델을 활용해 다른 **업무 자동화** 도구를 만드는 연습 기회가 될 것입니다. 또한 여러분의 업무 환경에 최적화된 자동화 도구를 만들 수도 있습니다.



04-1 PDF 문서 전처리하기

04-2 논문을 요약하는 AI 연구원 완성하기



04-1 PDF 문서 전처리하기

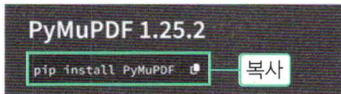
PDF 파일로 저장되어 있는 문서를 GPT와 같은 언어 모델을 이용해 작업하려면 **문서를 전처리하는 과정이 필요합니다**. PyMuPDF 패키지를 사용해 PDF 파일에서 텍스트를 추출할 수 있지만 텍스트로 변환할 때 페이지 번호나 헤더, 푸터 등의 정보가 섞일 수 있습니다. 이 절에서는 이러한 요소들을 처리하는 PDF 파일 텍스트 전처리 과정을 다루겠습니다.

Do it! 실습 PDF 파일을 텍스트 파일로 변환하기

■ 결과 파일: chap04/sec02/pdf_to_txt.py

1. PyMuPDF 가이드 문서에 따라 터미널 창에 **pip install PyMuPDF**를 입력하여 가상 환경(venv) 내에 설치합니다.

◆ PyMuPDF 가이드 문서 웹 페이지: <https://pypi.org/project/PyMuPDF/>



```
(venv) > pip install PyMuPDF
```

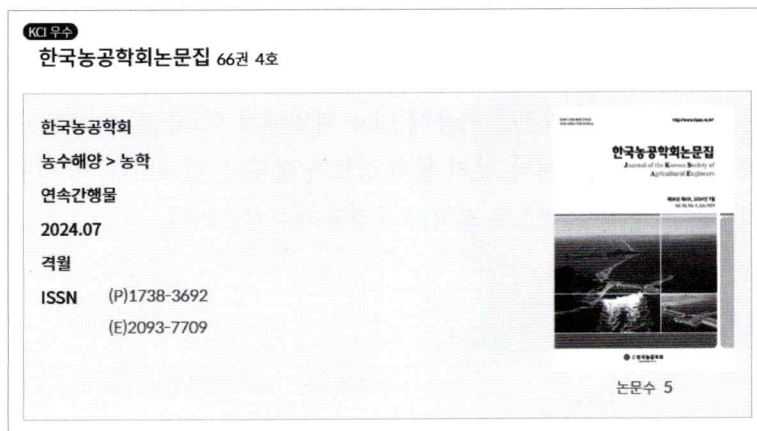
입력

2. 이제 요약할 PDF 파일을 준비합니다. PDF 파일은 다음처럼 텍스트 영역이 선택되는 문서를 사용해야 합니다. 이미지로 스캔한 PDF 파일은 PyMuPDF를 사용하는 이번 예제에 적합하지 않습니다.

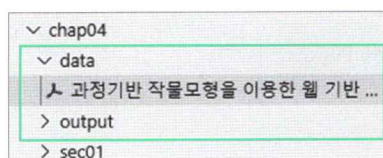
1. 서론

우리나라 밀 소비량은 지속적으로 증가하고 있지만 자급률은 1% 수준밖에 되지 않아 안정적인 생산기반을 마련할 필요가 제기되고 있다. 쌀과 밀, 서류, 콩 등을 포함한 우리나라 1인당 양곡소비량은 2022년 기준 총 125.4 kg이며, 이 중에서 쌀이 약 45.2% (56.7 kg), 그다음으로 밀이 30.3% (38.0 kg)를 차지한다. 2022년 1인당 밀 소비량을 기준으로 밀 소비량은 2012년 (32.9 kg/인)에 비해 연간 0.5%씩 증가하고 있다 (MAFRA, 2023a). 반면 식량자급률 (사료용 제외)은 2022년 기준 1.3%를 밑도는 수준이다 (MAFRA, 2023b). 이러한 문제

저는 한국농공학회(ksae.re.kr)에서 논문집 2024년 4호에 수록된 ‘과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원시스템 설계 및 구축’이라는 논문을 내려받아 활용하였습니다. 여러분은 원하는 PDF 문서를 사용하여 진행하면 됩니다.



3. 내려받은 PDF 파일을 사용하기 위해 chap04/data 폴더를 만들어 파일을 담습니다.



4. chap04 폴더에 새로운 파일을 만들고 PDF에서 텍스트를 잘 읽어 오는지 다음 코드를 작성해서 확인해 봅시다.

PDF에서 텍스트 추출하기
pdf_to_txt.py

```

import pymupdf
import os

pdf_file_path = "chap04/data/과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원 시스템 설계 및 구축.pdf"
doc = pymupdf.open(pdf_file_path)

full_text = ''

for page in doc: # 문서 페이지 반복
    text = page.get_text() # 페이지 텍스트 추출
    full_text += text
    
```

```
pdf_file_name = os.path.basename(pdf_file_path)
pdf_file_name = os.path.splitext(pdf_file_name)[0] # 확장자 제거

txt_file_path = f"chap04/output/{pdf_file_name}.txt"
with open(txt_file_path, 'w', encoding='utf-8') as f:
    f.write(full_text)
```

- ❶ 내려받은 PDF 파일을 PyMuPDF를 사용해 열면 PyMuPDF는 페이지별로 내용을 읽어 옵니다.
- ❷ doc 객체에서 각 페이지를 반복하여 텍스트를 추출하고, 이를 full_text라는 비어 있는 변수에 한 페이지씩 추가합니다.
- ❸ PDF 파일을 텍스트 파일로 저장하기 위해 원본 PDF 파일의 이름을 추출합니다.
- ❹ output 폴더에 full_text 내용을 텍스트 파일 형식으로 저장합니다.

5. chap04 폴더 안에 파일을 저장할 output 폴더를 만듭니다. 코드를 실행하고 저장된 파일을 열어 내용을 확인합니다. 전반적으로 텍스트를 잘 추출한 것 같지만 중간에 문맥과 관계없는 내용도 들어 있습니다.

PDF에서 텍스트 추출한 결과

■ 과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원 시스템 설계 및 구축.txt

(... 생략 ...)

밀과 같이 대부분 노지에서 재배하는 작물은 시설재배와 다르게 기상환경 조절이 어렵고 돌발 재해 및 병해충 피해와 같은 외부적인 요인에 따라 생산량에 큰 영향을 미친다 (Kang et al., 2015).

(... 생략 ...)

Received: December 28, 2023

Revised: May 02, 2024

Accepted: May 08, 2024

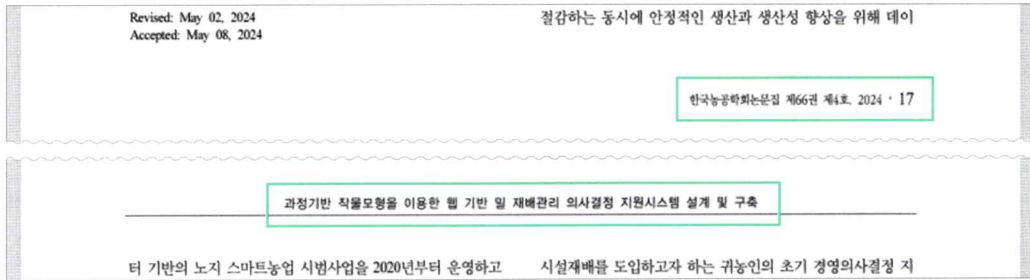
과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원시스템 설계 및 구축

18 • Journal of the Korean Society of Agricultural Engineers, 66(4), 2024. 7

터 기반의 노지 스마트농업 시범사업을 2020년부터 운영하고 있다. 한 예로, 경북 안동 사과 주산지와 충북 괴산 콩 주산지를 대상으로 관수·관비 등 ICT 기자재와 스마트 농기계 (드론, 무인트랙터 등)를 지원하고, 기존 농산물산지유통시설

(... 생략 ...)

중간에 페이지 번호와 학회지 이름, 논문 제목이 같이 끼어 있는 이유는 다음과 같이 페이지 헤더와 푸터 부분의 텍스트가 같이 포함되었기 때문입니다.



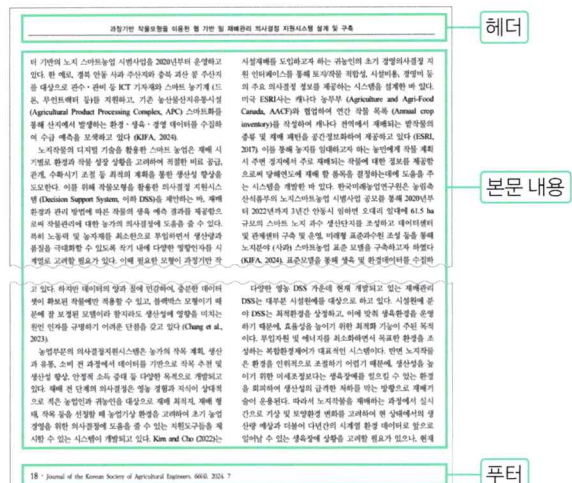
GPT가 잘 이해해서 요약하길 바라면서 남겨 둘 수도 있지만 전처리를 해준다면 더 나은 결과를 얻을 수 있겠죠. 다음 실습에서는 PDF를 전처리하는 방법을 알아보겠습니다.

Do it! 실습 PDF 파일 전처리 하기

■ 결과 파일: sec01/pdf_without_header_footer.py

앞선 실습에서 보았듯이 PDF 파일을 텍스트로 변환하면 일반적으로 읽는 본문뿐만 아니라 PDF의 모든 텍스트가 일괄적으로 변환됩니다. 제가 사용한 PDF 파일은 텍스트 위주의 단순한 구성이라 비교적 잘 변환되었지만, 페이지 상단과 하단에 부가 내용이 많은 문서는 GPT 같은 언어 모델이 제대로 처리하지 못할 수도 있습니다. 이런 경우 전처리가 필요합니다.

페이지의 상단과 하단, 즉 헤더와 푸터의 내용을 제외하고 본문 내용만 추출하고 싶다면 PDF에서 위와 아래 영역을 제외하고 텍스트를 읽도록 설정할 수 있습니다.



1. 다음은 PDF 파일에서 헤더와 푸터를 제외하고 텍스트를 추출하는 코드입니다. `full_text`에 헤더와 푸터 영역을 제외한 나머지 부분의 텍스트만 추가합니다. 헤더와 푸터의 높이는 모두 80으로 설정했습니다. 페이지의 크기는 `page.rect`를 사용해 확인한 후 폭(`rect.width`)과 높이(`rect.height`)를 이용해 텍스트를 추출할 영역을 `clip`으로 지정합니다. `full_text`에 덧붙일 때 `'\n-----\n'`으로 줄 바꿈 표시를 해서 페이지를 구분했습니다.

PDF의 헤더와 푸터는 제외하고 읽기

pdf_without_header_footer.py

```
import pymupdf
import os

pdf_file_path = "chap04/data/과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원 시스템 설계 및 구축.pdf"
doc = pymupdf.open(pdf_file_path)

header_height = 80
footer_height = 80

full_text = ''

for page in doc:
    rect = page.rect # 페이지 크기 가져오기

    header = page.get_text(clip=(0, 0, rect.width, header_height))
    footer = page.get_text(clip=(0, rect.height - footer_height, rect.width, rect.height))
    text = page.get_text(clip=(0, header_height, rect.width, rect.height - footer_height))

    full_text += text + '\n-----\n'

# 파일명만 추출
pdf_file_name = os.path.basename(pdf_file_path)
pdf_file_name = os.path.splitext(pdf_file_name)[0] # 확장자 제거

txt_file_path = f'chap04/output/{pdf_file_name}_with_preprocessing.txt'

with open(txt_file_path, 'w', encoding='utf-8') as f:
    f.write(full_text)
```

2. 코드를 실행하고 output 폴더에 저장된 결과를 보면 페이지 하단의 푸터 내용과 페이지 상단에 있던 페이지 번호와 논문 제목 등을 제외하고 본문 내용만 잘 입력되어 있습니다.

PDF 파일을 전처리하여 저장한 결과

■ 과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원 시스템 설계 및 구축_with_preprocessing.txt

(... 생략 ...)

기상 및 토양환경 변화를 고려하여 현 상태에서의 생산량 예상과 더불어 다년간의 시계열 환경 데이터로 앞으로 일어날 수 있는 생육장애 상황을 고려할 필요가 있으나, 현재 ... 생략 ...

김솔희, 석승원, 청리광, 장태일, 김태곤

한국농공학회논문집 제66권 제4호, 2024 • 19

관련 시스템을 찾아보기 어려운 실정이다.

본 연구에서는 밀을 대상으로 재배관리 의사결정 지원시스템을 웹 기반으로 설계하고 구축하고자 한다. 과정기반 모형을 이용하여 기상여건을 평가하고, 향후 기상을 고려하여 ... 생략 ...

이처럼 전처리 방식에 따라 GPT에게 제공하는 텍스트의 품질이 달라집니다. 전처리가 잘 되어 있다면 고급 모델인 gpt-4o를 사용할 필요 없이 비용이 더 저렴한 gpt-4o-mini를 활용해도 충분히 원하는 결과를 얻을 수 있습니다.

04-2 논문을 요약해 주는 AI 연구원 완성하기

PDF 문서를 전처리하고 텍스트로 변환했으므로 언어 모델을 활용해 원하는 작업을 요청할 수 있습니다. 이 절에서는 텍스트 파일의 내용을 요약해 보겠습니다.

Do it! 실습 텍스트 요약 프롬프트 만들기

■ 결과 파일: sec02/summary.py

이번 예제에서는 앞서 변환한 텍스트 파일을 활용해 문서 내용을 요약해 봅시다.

1. chap04 폴더에 summary.py 파일을 새로 만들고 텍스트 파일을 요약하는 코드를 작성합니다. 03-2절에서 작성한 multi_turn.py의 코드와 크게 다르지 않습니다.

텍스트 파일 읽고 요약하기

summary.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os
```

```
load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
```

```
def summarize_txt(file_path: str): ①
    client = OpenAI(api_key=api_key)
```

```
    with open(file_path, 'r', encoding='utf-8') as f: ②
        txt = f.read()
```

```
    system_prompt = f'''
    너는 다음 글을 요약하는 봇이다. 아래 글을 읽고, 저자의 문제 인식과 주장을 파악하고, 주요 내용을 요약하라.
```

```
    작성해야 하는 포맷은 다음과 같다.
```

```
    # 제목
```

```

## 저자의 문제 인식 및 주장 (15문장 이내)

## 저자 소개

===== 이하 텍스트 =====

{ txt }
'''

print(system_prompt)
print('=====')

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.1,
    messages=[
        {"role": "system", "content": system_prompt},
    ]
)

return response.choices[0].message.content

if __name__ == '__main__':
    file_path = './chap04/output/과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원
시스템 설계 및 구축_with_preprocessing.txt

    summary = summarize_txt(file_path)
    print(summary)

    with open('./chap04/output/crop_model_summary.txt', 'w', encoding='utf-8') as f:
        f.write(summary)

```

- ❶ 먼저 summarize_txt 함수를 만들고 file_path를 매개변수로 받게 합니다.
- ❷ 인자로 받은 file_path의 내용을 읽어 txt 변수에 담습니다.
- ❸ system_prompt에는 요약 방법이 담겨 있습니다. 제목을 먼저 쓰고 저자의 문제 인식과 주장을 15문장 이내로 요약하라고 주문한 뒤 저자 소개를 추가하도록 했습니다. 그리고 마지막에 요약을 위해 필요한 txt 내용을 추가합니다.
- ❹ 오픈AI의 gpt-4o에 요약을 요청하는 단계입니다. 여기서는 대화 형식으로 멀티턴을 할 필요가 없으므로 system_prompt의 내용을 "role": "system" 형식으로 넘겨줍니다.
- ❺ 코드의 메인 부분에서 summarize_txt 함수를 실행한 결과를 output 폴더에 텍스트 파일로 저장합니다.

2. 코드를 실행하고 output 폴더에 생성된 파일을 열어 보면 다음과 같이 요청한 형태로 문서가 잘 요약된 것을 확인할 수 있습니다.

파일 출력 결과

./output/crop_model_summary.txt

과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원시스템 설계 및 구축

저자의 문제 인식 및 주장

저자는 한국의 밀 소비량이 증가하고 있지만 자급률이 매우 낮아 안정적인 생산 기반이 필요하다고 인식하고 있다. 이를 해결하기 위해 정부는 밀 산업 발전을 위한 법적 토대를 마련했지만, 밀의 자급률을 높이기 위해서는 수입밀과의 가격 및 품질 경쟁력을 확보해야 한다고 주장한다. 특히, 노지에서 재배되는 밀은 기상 환경의 영향을 크게 받기 때문에, 디지털 기술을 활용한 스마트 농업이 필요하다고 강조한다. 저자는 과정 기반 작물모형인 APSIM을 활용하여 웹 기반의 밀 재배관리 의사결정 지원시스템을 설계하고 구축하였다. 이 시스템은 기상 데이터를 기반으로 밀의 잠재 수확량을 예측하고, 농가가 관수 및 비료 처방에 대한 의사결정을 지원할 수 있도록 설계되었다. 저자는 이 시스템이 전국의 기상 데이터와 토양 환경 정보를 활용하여 다양한 지역에 적용 가능하며, 향후 밀 생산량 및 품질 증대를 위한 의사결정 지원시스템으로 확장될 수 있을 것으로 기대한다.

저자 소개

김솔희, 석승원, 청리광, 장태일, 김태곤은 전북대학교 스마트팜 및 농업과학기술 연구소에 소속된 연구자들로, 디지털 농업 모델 개발 및 실증 연구를 수행하고 있다. 이들은 밀 생산성 향상을 위한 디지털 농업 모델 개발을 목표로 연구를 진행하고 있으며, 특히 과정기반 작물모형을 활용한 의사결정 지원시스템 개발에 주력하고 있다.

여기에서 사용한 시스템 프롬프트는 해당 글의 요약에 맞춘 예시입니다. 여러분이 원하는 형태가 있다면 사용하는 문서의 내용, 원하는 출력 형식을 시스템 프롬프트에 작성해 적용하면 됩니다.

Do it! 실습 PDF 내용 요약하여 출력하기

결과 파일: sec02/pdf_summary.py

모든 기능 개발은 끝났으므로 PDF 파일만 입력하면 요약 결과를 저장하는 AI 연구원을 완성해 보겠습니다. 앞에서 작성한 summary.py 파일에 앞에서 만든 PDF 파일을 텍스트로 변환해 저장하는 pdf_without_header_footer.py 코드를 함수로 만들어 추가하기만 하면 됩니다.

다음 코드에서 새로운 것은 없습니다. 함수 `pdf_to_text`를 만들고 04-1장에서 만들었던 `pdf_without_header_footer.py` 파일의 내용을 그대로 가져옵니다. `summarize_pdf` 함수는 `pdf_to_text` 함수를 실행한 결과를 예전에 만들어 두었던 `summarize_txt` 함수에 넣어 실행하는 역할입니다.

PDF 파일 경로 입력하면 요약하는 기능 만들기

pdf_summary.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os
import pymupdf

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')

def pdf_to_text(pdf_file_path: str):
    doc = pymupdf.open(pdf_file_path)

    header_height = 80
    footer_height = 80

    full_text = ''

    for page in doc:
        rect = page.rect # 페이지 크기 가져오기

        header = page.get_text(clip=(0, 0, rect.width, header_height))
        footer = page.get_text(clip=(0, rect.height - footer_height, rect.width, rect.
height))
        text = page.get_text(clip=(0, header_height, rect.width, rect.height - foot-
er_height))

        full_text += text + '\n-----\n'

    # 파일명만 추출
    pdf_file_name = os.path.basename(pdf_file_path)
    pdf_file_name = os.path.splitext(pdf_file_name)[0] # 확장자 제거

    txt_file_path = f'chap04/output/{pdf_file_name}_with_preprocessing.txt'

    with open(txt_file_path, 'w', encoding='utf-8') as f:
        f.write(full_text)
```

```

    return txt_file_path

def summarize_txt(file_path: str):
    (... 생략 ...)

def summarize_pdf(pdf_file_path: str, output_file_path: str):
    txt_file_path = pdf_to_text(pdf_file_path)
    summary = summarize_txt(txt_file_path)

    with open(output_file_path, 'w', encoding='utf-8') as f:
        f.write(summary)

if __name__ == '__main__':
    pdf_file_path = "chap04/data/과정기반 작물모형을 이용한 웹 기반 밀 재배관리 의사결정 지원시
스텨 설계 및 구축.pdf"
    summarize_pdf(pdf_file_path, 'chap04/output/crop_model_summary2.txt')

```

이렇게 PDF 파일의 내용을 요약해 주는 AI 연구자를 완성했습니다. 이제 `summarize_pdf` 함수를 사용하여 여러 PDF 파일을 쉽게 요약할 수 있습니다.